

Guía de uso de estilos, temas y componentes

Plataforma Leo y Escribo

Documento técnico para mantener una interfaz consistente, reutilizable, accesible y adaptable en las pantallas de la plataforma.

Proyecto de horas de extensión

Índice

1	Objetivo y alcance	2
2	Organización de los estilos	2
2.1	Archivos principales	2
2.2	Carga global	2
3	Variables y tokens globales	3
3.1	Colores semánticos	3
3.2	Tipografía	4
3.3	Espaciado, radios, sombras y transición	4
4	Temas por unidad y lección	4
4.1	Asignación centralizada	5
4.2	Cómo registrar una nueva lección	5
5	Componentes reutilizables	6
5.1	Button	6
5.2	Card	6
5.3	Componentes de progreso	7
5.4	Componentes de navegación	8
6	Ejemplo de uso para una lección	8
6.1	Estilo específico de la pantalla	10
7	Ejemplo de uso para una actividad	10
7.1	CSS mínimo de la actividad	12
8	Uso de estilos y componentes en la landing	12
8.1	Composición con componentes existentes	12
8.2	Responsabilidad de landing.css	14
9	Accesibilidad y uso responsable del color	15
10	Diseño responsive	15
11	Flujo recomendado para crear una pantalla	15
12	Errores que deben evitarse	16
13	Lista de verificación	16
14	Conclusión	17

1. Objetivo y alcance

Esta guía documenta brevemente cómo utilizar los estilos, las variables CSS, los temas de color y los componentes reutilizables creados para la plataforma **Leo y Escribo**. Su propósito es que las nuevas pantallas conserven la misma identidad visual y que no sea necesario repetir código para botones, tarjetas, navegación o indicadores de progreso.

La guía también incluye :

1. Ejemplo completo de una lección con tema asignado por unidad.
2. Ejemplo completo de una actividad con audio, opciones y retroalimentación.

2. Organización de los estilos

2.1. Archivos principales

Archivo	Responsabilidad
<code>src/styles/global.css</code>	Tokens globales, reset, tipografía base y estilos compartidos de componentes.
<code>src/styles/lesson-themes.css</code>	Variables semánticas y temas de color reutilizados por las unidades.
<code>src/styles/themes.css</code>	Variables complementarias, principalmente estados hover, active y disabled.
<code>src/styles/typography.css</code>	Reglas especializadas para letras, sílabas e instrucciones.
<code>src/styles/reading.css</code>	Presentación de actividades de lectura.
<code>src/styles/syllables.css</code>	Presentación de ejercicios con sílabas.
<code>src/styles/selection.css</code>	Presentación de actividades de selección.
<code>src/styles/completion.css</code>	Presentación de actividades para completar palabras.
<code>src/styles/landing.css</code>	Distribución exclusiva de la landing y sus dos tarjetas de proyecto.

2.2. Carga global

Los archivos generales se importan una sola vez desde `src/main.jsx`. Las páginas no deben volver a importar estos estilos, pues hacerlo genera dependencias innecesarias y puede alterar el orden de la cascada.

Listing 1: Importación global de estilos en main.jsx

```
1 import './styles/global.css'
2 import './styles/lesson-themes.css'
3 import './styles/typography.css'
4 import './styles/reading.css'
5 import './styles/syllables.css'
6 import './styles/selection.css'
7 import './styles/completion.css'
8 import './styles/themes.css'
```

Los estilos exclusivos de una página sí se importan directamente desde ella. Por ejemplo, la landing utiliza:

```
1 import '../styles/landing.css'
```

Importante: Una página nueva debe reutilizar primero las clases y componentes existentes. Solo se crea un CSS específico cuando la pantalla necesita una distribución que todavía no existe.

3. Variables y tokens globales

Los tokens son variables que guardan decisiones visuales comunes. Al utilizarlos, un cambio en la paleta, tipografía o espaciado puede aplicarse a toda la plataforma desde un solo lugar.

3.1. Colores semánticos

Variable	Uso recomendado
-color-action	Acciones principales, enlaces y navegación activa.
-color-action-secondary	Apoyo visual del azul principal.
-color-action-soft	Fondos suaves para controles de audio o destacados.
-color-success / -color-correct	Respuesta o resultado correcto.
-color-support	Apoyo visual positivo sin representar una respuesta correcta.
-color-retry	Aviso de reintento o respuesta que debe revisarse.
-color-reward	Estrellas, premios e insignias.
-color-accent	Elementos secundarios destacados.
-color-background / -color-bg	Fondo general de las páginas.
-color-surface / -color-bg-card	Superficie de tarjetas y contenedores.
-color-text	Texto principal.
-color-text-alt	Texto secundario o descriptivo.
-color-border	Bordes y separadores neutros.

Los nombres describen la **función** y no solamente el color. Por ejemplo, una respuesta correcta debe usar `-color-correct`, aunque posteriormente se cambie el tono de verde.

Listing 2: Uso correcto de variables semánticas

```
1 .activity-feedback--correct {
2   padding: var(--space-md);
3   color: #ffffff;
4   background-color: var(--color-correct);
5   border-radius: var(--radius-md);
6 }
```

3.2. Tipografía

Variable	Uso
<code>-font-content</code>	Andika: letras, sílabas, palabras, lecturas e instrucciones.
<code>-font-ui</code>	Nunito: títulos, botones, menús y navegación.
<code>-font-alt</code>	Lexend: alternativa para instrucciones secundarias.
<code>-font-size-xs / sm</code>	Etiquetas y texto de apoyo.
<code>-font-size-md</code>	Texto principal.
<code>-font-size-lg / xl / 2xl</code>	Subtítulos y títulos.
<code>-font-size-syllable</code>	Letras y sílabas presentadas en gran tamaño.

La fuente base del contenido es Andika. Los encabezados y componentes de interfaz utilizan Nunito automáticamente mediante las reglas globales.

3.3. Espaciado, radios, sombras y transición

- Espaciado: `-space-xs`, `-space-sm`, `-space-md`, `-space-lg`, `-space-xl` y `-space-2xl`.
- Radios: `-radius-sm`, `-radius-md`, `-radius-lg` y `-radius-full`.
- Sombras: `-shadow-sm`, `-shadow-md` y `-shadow-btn`.
- Transición compartida: `-transition`.

Listing 3: Combinación de tokens globales

```

1 .example-panel {
2   padding: var(--space-lg);
3   color: var(--color-text);
4   background-color: var(--color-bg-card);
5   border: 1px solid var(--color-border);
6   border-radius: var(--radius-lg);
7   box-shadow: var(--shadow-md);
8 }
```

No se recomienda crear valores como `padding: 23px` o un color hexadecimal nuevo si ya existe un token equivalente. Esto evita pequeñas diferencias visuales entre páginas.

4. Temas por unidad y lección

Los temas permiten diferenciar grupos de lecciones sin asignar un color distinto a cada letra. Las unidades reutilizan los colores de manera ordenada:

Unidades	Tema	Descripción
1 y 4	Azul	Azul principal con apoyo verde.
2 y 5	Morado	Morado y acento suave; las acciones mantienen significado semántico.
3 y 6	Cielo y menta	Azul secundario, verde de apoyo y acento suave.

Cada tema define las siguientes variables locales:

- `-lesson-primary`: color representativo principal.
- `-lesson-secondary`: color secundario del tema.
- `-lesson-accent`: detalle visual complementario.
- `-lesson-surface`: superficie de la lección.
- `-lesson-border`: bordes de tarjetas de lección.
- `-lesson-text`: texto dentro de la lección.
- `-lesson-background`: fondo de la pantalla.

4.1. Asignación centralizada

La relación entre el identificador de una lección y su unidad se guarda en `src/data/lessonColors.js`. Las páginas no deben escoger el tema con condiciones propias ni colocar directamente una clase de unidad.

```
1 import { getLessonThemeClass } from '../data/lessonColors'
2
3 const themeClass = getLessonThemeClass(lessonId)
4
5 <main className={`page lesson-detail-page ${themeClass}`}>
6   {/* Contenido de la lección */}
7 </main>
```

Si el identificador no está registrado, la función devuelve `lesson-theme-default`. Así la página continúa siendo legible y no queda sin estilos.

4.2. Cómo registrar una nueva lección

1. Definir un identificador estable para la ruta, por ejemplo `repaso-vocales`.
2. Agregarlo una sola vez en `LESSON_UNIT_MAP`.
3. Asignar el número de unidad correspondiente.
4. Usar `getLessonThemeClass(lessonId)` en la pantalla.
5. Evitar crear un color exclusivo para esa lección.

```
1 export const LESSON_UNIT_MAP = {
2   vocales: 1,
3   m: 1,
4   'repaso-vocales': 1,
5   c: 2,
6   b: 3,
7 }
```

5. Componentes reutilizables

5.1. Button

El componente se importa desde `src/components/common/Button.jsx`. Puede renderizar un botón normal o un enlace interno cuando recibe la propiedad `to`.

Propiedad	Descripción
<code>children</code>	Texto o contenido visible.
<code>to</code>	Ruta interna; al usarla se renderiza un <code>Link</code> .
<code>type</code>	Tipo HTML del botón; por defecto es <code>button</code> .
<code>variant</code>	Variante visual semántica.
<code>size</code>	<code>small</code> , <code>medium</code> o <code>large</code> .
<code>icon</code>	Componente de ícono de Lucide.
<code>iconPosition</code>	Posición <code>left</code> o <code>right</code> .
<code>fullWidth</code>	Ocupa el ancho disponible.
<code>disabled</code>	Desactiva la interacción.
<code>className</code>	Permite agregar una clase específica sin reemplazar las globales.

Variantes disponibles: `primary`, `secondary`, `correct`, `accent`, `support`, `retry`, `reward` y `audio`.

Listing 4: Ejemplos de Button

```
1 import { Volume2 } from 'lucide-react'
2 import Button from '../components/common/Button'
3
4 <Button to="/lecciones" variant="primary">
5   Ver lecciones
6 </Button>
7
8 <Button
9   variant="audio"
10  icon={Volume2}
11  onClick={playAudio}
12 >
13   Escuchar audio
14 </Button>
15
16 <Button variant="secondary" fullWidth disabled>
17   Próximamente
18 </Button>
```

Se debe usar `Button` en lugar de escribir manualmente combinaciones como `className="button primary"`. De esta manera se mantienen el tamaño táctil, la forma, la tipografía y los estados deshabilitados.

5.2. Card

El componente se importa desde `src/components/common/Card.jsx`. Sirve para agrupar información de lecciones, actividades, proyectos u opciones.

Propiedad	Descripción
<code>title</code>	Título de la tarjeta.
<code>subtitle</code>	Texto secundario breve.
<code>description</code>	Descripción del contenido.
<code>imageSrc / imageAlt</code>	Imagen y texto alternativo.
<code>icon</code>	Ícono decorativo.
<code>to</code>	Convierte la tarjeta en enlace interno.
<code>selected</code>	Marca una tarjeta seleccionada.
<code>clickable</code>	Aplica comportamiento visual interactivo.
<code>footer</code>	Contenido fijo al final, normalmente un botón.
<code>children</code>	Contenido JSX adicional.
<code>className</code>	Clase específica de la página.

Listing 5: Tarjeta de una lección

```

1 import { BookOpen } from 'lucide-react'
2 import Card from '../components/common/Card'
3
4 <Card
5   to="/leccion/m"
6   clickable
7   icon={BookOpen}
8   title="Letra M"
9   description="Escucha y practica el sonido de la letra."
10 />

```

Listing 6: Tarjeta con imagen y pie

```

1 <Card
2   imageSrc="/images/lenguaje.png"
3   imageAlt="Niños aprendiendo lenguaje y leyendo"
4   title="Leo y Aprendo"
5   description="Practica la lectura y la escritura."
6   footer={
7     <Button to="/login" fullWidth>
8       Ingresar
9     </Button>
10  }
11 />

```

5.3. Componentes de progreso

ProgressBar. Muestra el progreso de un valor respecto a un máximo. El componente limita el resultado al rango de 0 a 100 y agrega los atributos de accesibilidad de una barra de progreso.

```

1 <ProgressBar
2   value={3}
3   max={5}
4   label="Actividades completadas"
5 />

```


StarsCounter. Presenta las estrellas obtenidas y el total disponible.

```
1 <StarsCounter current={4} total={6} label="Estrellas" />
```

LessonProgress. Muestra una lista de pasos y diferencia el actual de los completados.

```
1 const steps = [  
2   { id: 'audio', label: 'Escuchar', completed: true },  
3   { id: 'practice', label: 'Practicar', completed: false },  
4 ]  
5  
6 <LessonProgress  
7   lessons={steps}  
8   currentLessonId="practice"  
9 />
```

5.4. Componentes de navegación

BackButton. Regresa a una ruta indicada o utiliza el historial del navegador cuando no recibe `to`.

```
1 <BackButton label="Volver a lecciones" to="/lecciones" />
```

BottomNav. Incluye por defecto las rutas Inicio, Lecciones y Progreso. Usa `NavLink` para marcar la opción activa.

```
1 <BottomNav />
```

También admite una lista personalizada mediante la propiedad `items`, siempre que cada elemento incluya `label`, `to` e `icon`.

6. Ejemplo de uso para una lección

El siguiente ejemplo muestra una pantalla que obtiene el identificador desde la ruta, aplica automáticamente el tema de la unidad y reutiliza los componentes de tarjeta, progreso, navegación y botones.

Listing 7: LessonExample.jsx

```
1 import { Volume2 } from 'lucide-react'  
2 import { useParams } from 'react-router-dom'  
3 import Button from '../components/common/Button'  
4 import Card from '../components/common/Card'  
5 import BackButton from '../components/navigation/BackButton'  
6 import ProgressBar from '../components/progress/ProgressBar'  
7 import { getLessonThemeClass } from '../data/lessonColors'  
8  
9 function LessonExample() {  
10   const { lessonId = 'vocales' } = useParams()  
11   const themeClass = getLessonThemeClass(lessonId)  
12  
13   const playIntroduction = () => {  
14     // Reproducir el audio correspondiente a la lección.
```

```
15 }
16
17 return (
18   <main
19     className={`page lesson-detail-page ${themeClass}`}
20     aria-labelledby="lesson-title"
21   >
22     <div className="container">
23       <BackButton to="/lecciones" />
24
25       <Card
26         title={`Lección: ${lessonId.toUpperCase()}`}
27         subtitle="Escucha, observa y practica"
28       >
29         <h1 id="lesson-title" className="lesson-letter">
30           M
31         </h1>
32
33         <p>
34           Escucha el sonido de la letra y completa
35           las actividades.
36         </p>
37
38         <Button
39           variant="audio"
40           icon={Volume2}
41           onClick={playIntroduction}
42           fullWidth
43         >
44           Escuchar sonido
45         </Button>
46
47         <ProgressBar
48           value={1}
49           max={4}
50           label="Progreso de la lección"
51         />
52
53         <Button
54           to={`/actividad/${lessonId}-silabas`}
55           variant="primary"
56           fullWidth
57         >
58           Practicar sílabas
59         </Button>
60       </Card>
61     </div>
62   </main>
63 )
64 }
65
66 export default LessonExample
```

6.1. Estilo específico de la pantalla

El CSS de la lección debe consumir las variables `--lesson-*`, no colores hexadecimales ni clases diferentes para cada letra.

Listing 8: Ejemplo de CSS temático

```
1 .lesson-detail-page {
2   color: var(--lesson-text);
3   background-color: var(--lesson-background);
4 }
5
6 .lesson-detail-page .app-card {
7   background-color: var(--lesson-surface);
8   border: 3px solid var(--lesson-border);
9 }
10
11 .lesson-letter {
12   color: var(--lesson-primary);
13   font-family: var(--font-content);
14   font-size: var(--font-size-syllable);
15   text-align: center;
16 }
```

Este patrón permite que la misma pantalla funcione para todas las lecciones. Lo único que cambia es el identificador y, por medio de este, la clase temática.

7. Ejemplo de uso para una actividad

El siguiente ejemplo presenta una actividad de selección. Incluye una instrucción, reproducción de audio, opciones, retroalimentación textual y navegación al terminar.

Listing 9: ActivityExample.jsx

```
1 import { useState } from 'react'
2 import { Volume2 } from 'lucide-react'
3 import Button from '../components/common/Button'
4 import Card from '../components/common/Card'
5 import BackButton from '../components/navigation/BackButton'
6 import StarsCounter from '../components/progress/StarsCounter'
7
8 function ActivityExample() {
9   const [selected, setSelected] = useState('')
10   const [result, setResult] = useState('')
11
12   const playAudio = () => {
13     // Reproducir la sílaba objetivo.
14   }
15
16   const checkAnswer = () => {
17     setResult(selected === 'ma' ? 'correct' : 'retry')
18   }
19
20   return (
21     <main className="page activity-page">
```

```

22     <div className="container">
23       <BackButton to="/leccion/m" />
24
25       <Card title="Escucha y selecciona">
26         <p id="activity-instruction">
27           Escucha la sílaba y elige la respuesta correcta.
28         </p>
29
30         <Button
31           variant="audio"
32           icon={Volume2}
33           onClick={playAudio}
34           fullWidth
35           aria-label="Escuchar nuevamente la sílaba"
36         >
37           Escuchar audio
38         </Button>
39
40         <div
41           className="activity-options"
42           aria-labelledby="activity-instruction"
43         >
44           {[ 'ma', 'me', 'mi' ].map((option) => (
45             <Button
46               key={option}
47               variant={
48                 selected === option ? 'accent' : 'secondary'
49               }
50               onClick={() => {
51                 setSelected(option)
52                 setResult('')
53               }}
54             >
55               {option}
56             </Button>
57           )))
58         </div>
59
60         {result === 'correct' && (
61           <p className="feedback-correct" role="status">
62             ¡Correcto! Seleccionaste la sílaba ma.
63           </p>
64         )}
65
66         {result === 'retry' && (
67           <p className="feedback-retry" role="status">
68             Inténtalo nuevamente. Escucha con atención.
69           </p>
70         )}
71
72         <Button
73           variant="primary"
74           onClick={checkAnswer}
75           disabled={!selected}

```

```

76         fullWidth
77     >
78         Comprobar respuesta
79     </Button>
80
81     <StarsCounter
82         current={result === 'correct' ? 1 : 0}
83         total={1}
84     />
85 </Card>
86 </div>
87 </main>
88 )
89 }
90
91 export default ActivityExample

```

7.1. CSS mínimo de la actividad

```

1 .activity-options {
2     display: grid;
3     grid-template-columns: repeat(3, minmax(0, 1fr));
4     gap: var(--space-md);
5 }
6
7 @media (max-width: 480px) {
8     .activity-options {
9         grid-template-columns: 1fr;
10    }
11 }

```

La selección se comunica por estilo y por el contenido del control. El resultado usa color y también un mensaje textual dentro de un elemento con `role="status"`.

8. Uso de estilos y componentes en la landing

La landing presenta únicamente dos tarjetas:

1. **Leo y Aprendo**, que permite ingresar al módulo de lenguaje.
2. **Matemáticas**, mostrado como proyecto próximo.

Las ilustraciones se guardan en:

```

1 public/images/lenguaje.png
2 public/images/mate.png

```

8.1. Composición con componentes existentes

Listing 10: WelcomePage.jsx

```

1 import Button from '../components/common/Button'
2 import Card from '../components/common/Card'

```

```
3 import '../styles/landing.css'
4
5 function WelcomePage() {
6   return (
7     <main className="page landing-page">
8       <section
9         className="landing-page__content"
10        aria-labelledby="landing-title"
11      >
12        <header className="landing-page__header">
13          <h1 id="landing-title">¡Bienvenido!</h1>
14          <p>Elige el proyecto al que deseas ingresar.</p>
15        </header>
16
17        <div className="landing-page__projects">
18          <Card
19            className="project-card project-card--language"
20            imageSrc="/images/lenguaje.png"
21            imageAlt="Niños aprendiendo lenguaje y leyendo"
22            title="Leo y Aprendo"
23            description="Practica la lectura y la escritura
24              con lecciones y actividades."
25            footer={
26              <Button to="/login" fullWidth>
27                Ingresar
28              </Button>
29            }
30          />
31
32          <Card
33            className={
34              'project-card project-card--math
35                project-card--upcoming'
36            }
37            imageSrc="/images/mate.png"
38            imageAlt="Niños aprendiendo matemáticas"
39            title="Matemáticas"
40            description="Aprende matemáticas mediante
41              actividades educativas."
42            footer={
43              <Button variant="secondary" fullWidth disabled>
44                Próximamente
45              </Button>
46            }
47          />
48        </div>
49      </section>
50    </main>
51  )
52 }
53
54 export default WelcomePage
```

8.2. Responsabilidad de landing.css

landing.css puede definir:

- ancho y separación de las dos columnas;
- tamaño de las imágenes;
- borde temático de cada proyecto;
- adaptación a una columna en pantallas pequeñas;
- un movimiento hover suave y respeto por `prefers-reduced-motion`.

No debe volver a definir la paleta, la tipografía, el botón ni la estructura interna general de `Card`.

Listing 11: Fragmento representativo de landing.css

```
1 .landing-page__content {
2   width: min(100%, 1200px);
3   margin-inline: auto;
4 }
5
6 .landing-page__projects {
7   display: grid;
8   grid-template-columns: repeat(2, minmax(0, 1fr));
9   gap: clamp(1.5rem, 3vw, 2.5rem);
10 }
11
12 .project-card {
13   min-height: 470px;
14   border: 3px solid transparent;
15 }
16
17 .project-card .app-card__image {
18   width: 100%;
19   height: clamp(240px, 22vw, 300px);
20   object-fit: cover;
21   border-radius: var(--radius-md);
22 }
23
24 .project-card--language {
25   border-color: var(--color-action);
26 }
27
28 .project-card--math {
29   border-color: var(--color-accent);
30 }
31
32 @media (max-width: 800px) {
33   .landing-page__projects {
34     grid-template-columns: 1fr;
35     max-width: 600px;
36     margin-inline: auto;
37   }
38 }
39
40 @media (prefers-reduced-motion: reduce) {
```

```
41 .project-card {  
42   transition: none;  
43 }  
44 }
```

La landing utiliza ilustraciones como elemento llamativo, pero mantiene el fondo limpio, dos opciones claras y animaciones discretas. Esto ayuda a conservar un diseño infantil sin agregar estimulación visual innecesaria.

9. Accesibilidad y uso responsable del color

1. Mantener una altura táctil mínima de 48 px en acciones principales.
2. No utilizar únicamente el color para comunicar correcto, incorrecto, seleccionado o deshabilitado.
3. Acompañar los estados con texto, ícono o audio.
4. Agregar texto alternativo útil a imágenes con información.
5. Usar texto alternativo vacío (`alt=""`) en imágenes puramente decorativas.
6. Agregar `aria-label` a controles cuyo propósito no sea evidente.
7. Conservar un indicador textual en barras de progreso.
8. Evitar animaciones constantes o movimientos fuertes.
9. Respetar `prefers-reduced-motion`.
10. Mantener instrucciones cortas, directas y legibles.

10. Diseño responsive

Las páginas deben comenzar con una disposición para pantallas pequeñas y crecer sin perder legibilidad. En el sistema actual:

- `.page` ofrece el espaciado general.
- `.container` limita el contenido educativo a un ancho legible.
- Las grillas deben cambiar a una columna cuando sus opciones se vuelvan demasiado estrechas.
- Las imágenes usan `object-fit: cover` para conservar la proporción visual de las tarjetas.
- `clamp()` permite tamaños flexibles con límites seguros.

11. Flujo recomendado para crear una pantalla

1. Identificar si la pantalla es global, una lección o una actividad.
2. Revisar si ya existe un componente que resuelva cada parte.
3. Construir la estructura con `Button`, `Card`, componentes de progreso y navegación.
4. Aplicar el tema mediante `getLessonThemeClass` si corresponde a una lección.

5. Utilizar tokens globales en cualquier CSS adicional.
6. Agregar solamente clases específicas necesarias para la distribución.
7. Probar estados normal, hover, selected, disabled, correct y retry.
8. Verificar computadora, tablet y celular.
9. Revisar navegación por teclado, etiquetas y mensajes de estado.

12. Errores que deben evitarse

Evitar	Usar en su lugar
Colores hexadecimales repetidos en cada página.	Variables <code>-color-*</code> y <code>-lesson-*</code> .
Crear un color diferente para cada letra.	Asignar la lección a una unidad en <code>LESSON_UNIT_MAP</code> .
Escribir botones con clases manuales diferentes.	Componente <code>Button</code> y su propiedad <code>variant</code> .
Repetir la estructura de una tarjeta.	Componente <code>Card</code> .
Importar todos los CSS en cada página.	Importación global en <code>main.jsx</code> ; importar solo CSS exclusivo.
Comunicar una respuesta únicamente con verde o naranja.	Color acompañado de texto, ícono o audio.
Agregar adornos y animaciones constantes.	Detalles moderados y movimiento opcional.
Modificar el componente global para resolver una sola pantalla.	Agregar una clase específica de página mediante <code>className</code> .

13. Lista de verificación

Antes de considerar terminada una pantalla:

- ☐ Reutiliza los componentes existentes.
- ☐ Usa variables CSS en lugar de nuevos valores repetidos.
- ☐ La tipografía corresponde al contenido o a la interfaz.
- ☐ Si es una lección, obtiene el tema desde `lessonColors.js`.
- ☐ Las acciones tienen texto comprensible.
- ☐ Los estados no dependen únicamente del color.
- ☐ Los botones son suficientemente grandes para interacción táctil.
- ☐ Las imágenes incluyen `alt` adecuado.
- ☐ La pantalla funciona con teclado.
- ☐ El diseño se adapta a computadora, tablet y celular.
- ☐ El movimiento se reduce cuando el dispositivo lo solicita.

14. Conclusión

El sistema de estilos de Leo y Escribo separa las decisiones generales de las necesidades particulares de cada pantalla. `global.css` concentra los tokens y los componentes visuales; `lesson-themes.css` asigna una apariencia coherente a las unidades; `lessonColors.js` relaciona cada lección con su tema; y los archivos específicos, como `landing.css`, se limitan a organizar el contenido propio de una vista.

Al seguir esta estructura, las nuevas lecciones y actividades pueden añadirse sin duplicar estilos, sin crear una paleta diferente para cada contenido y sin perder consistencia visual o accesibilidad.